

Software Metrics, Analytics, and Quality

Docente: Carmine Gravino
Anno Accademico: 2025/2026

Software Metrics, Analytics, and Quality: Perché?

Come possiamo dire che un prodotto software è di qualità?

– Attributi di qualità

“Non puoi controllare quello che non puoi misurare” (cit. Tom De Marco)

Perché è importante?

Progetti	Con misurazioni	Senza misurazioni
puntuali	75%	45%
In ritardo	20%	40%
cancellati	5%	15%
Rimozione difetti	95%	<85%
Stima risorse necessarie	Accurata	Ottimistica
Soddisfazione del cliente	Alta	Bassa
Morale del team	Alto	Basso

Software Metrics, Analytics, and Quality: Perché?

Le entità visibili e misurabili sono i processi (le attività), le risorse impiegate e alcuni attributi dei prodotti

Attributi di qualità interni ed esterni

esterni: visibili all'utente

interni: visibili ai costruttori

Esempi di misure usate in SW Eng:

codice	lunghezza	LOC
	funzionalità	Function Points
	complessità	Indice McCabe
specifiche	lunghezza	#pagine
	riuso	#pagine
codifica	sforzo	Mesi/persona
testing	Fase rilevazione	%difetti trovati
	volume	#test schedulati
manutenzione	costo	€

Il metodo sperimentale

Introdurre metodologie e strumenti per:

- Definire, preparare e condurre studi empirici nell'ambito dell'ingegneria del software
- Costruire modelli statistici a partire da dati raccolti su
Prodotti software
Processi software

Il metodo sperimentale

Ma a cosa serve davvero?

Valutazione di tecniche, metodologie e strumenti applicati:

- Da partecipanti
- Su prodotti software
- Su processi software

Il metodo sperimentale

Esempi di domanda di ricerca da investigare:

Come selezionare il tool di sviluppo che meglio risponda alle mie esigenze?

L'utilizzo di un particolare tool di visualizzazione del codice è utile per i manutentori?

La manutenibilità del codice migliora utilizzando i design pattern?

L'utilizzo di tecniche di ispezione del codice riduce la difettosità dello stesso?

L'utilizzo degli stereotipi aumenta la comprensibilità dei diagrammi UML?

Il metodo sperimentale

Esempi di domanda di ricerca da investigare:

Strumenti a supporto della tracciabilità tra codice e requisiti migliorano la leggibilità del codice?

COSMIC è un buon metodo di misurazione funzionale da impiegare per stimare la taglia e lo sforzo di sviluppo di applicazioni software?

Posso stimare la difettosità del mio codice a partire da metriche OO?

Tecniche di prioritizzazione delle test suite riducono il costo dell'attività di testing senza influenzarne massicciamente l'efficacia?

Quale tecnica di clone detection funziona meglio?

- La più adatta per programmi OO?
- La più costosa?

Il metodo sperimentale

Quando applico queste tecniche: il mondo della produzione

Una decisione sbagliata può essere fonte di costi insostenibili

La metodologia sbagliata potrebbe portare alla produzione di un prodotto di qualità non buona

Occorre quindi poter prendere le giuste decisioni in tempi brevi e con costi tutto sommato ridotti

Utilizzare tecniche predittive per:

Stimare costi

Pianificare i progetti

Il metodo sperimentale

Quando applico queste tecniche: il mondo della ricerca

Ho sviluppato un nuovo approccio/metodo/tool e sono interessato a valutarne le performance

- Qual è la precisione?
- Funziona meglio di altri?
- Meglio con o senza?

Sono interessato a costruire modelli che legano diverse variabili

- Difettosità in funzione di caratteristiche strutturali del codice
- Sforzo in termine di numero di casi d'uso
- Manutenibilità in termine di caratteristiche di un class diagram

Vorrei poter confrontare tecniche esistenti, che altri hanno proposto senza confrontare

Sono interessato a replicare uno studio empirico esistente per ottenere ulteriore evidenza

Data extraction and analysis

Estrarre e astrarre dati da artefatti software e repository come:

- codice sorgente,
- sistemi di controllo delle versioni e revisioni,
- mailing list e discussioni, e issue-tracker e problemi.

Recuperare comportamenti e processi di sviluppo software da prove o per testare empiricamente ipotesi sullo sviluppo software.

Modelli di previsione sfruttando i dati estratti dal software del repository

Software Metrics, Analytics, and Quality: Overview del Corso

M1. Introduzione alla misurazione

M2. Metodi di Misurazione Funzionale

M3. Modelli di qualità del software

M4. Processo sperimentale

M5. Analisi dei dati e Modelli di predizione

Software Metrics, Analytics, and Quality: Overview del Corso

M1. Introduzione alla misurazione

M2. Metodi di Misurazione Funzionale

M3. Modelli di qualità del software

M4. Processo sperimentale

M5. Analisi dei dati e Modelli di predizione

- Metriche e Misure nell'Ingegneria del Software
- Uso del Paradigma Goal - Question- Metric
- Misurare per controllare e validare

Software Metrics, Analytics, and Quality: Overview del Corso

M1. Introduzione alla misurazione

M2. Metodi di Misurazione Funzionale

M3. Modelli di qualità del software

M4. Processo sperimentale

M5. Analisi dei dati e Modelli di predizione

- Function Point Analysis
- COSMIC

Software Metrics, Analytics, and Quality: Overview del Corso

M1. Introduzione alla misurazione

M2. Metodi di Misurazione Funzionale

M3. Modelli di qualità del software

M4. Processo sperimentale

M5. Analisi dei dati e Modelli di predizione

- Il modello ISO/IEC 9126
- Il modello ISO 25010

Software Metrics, Analytics, and Quality: Overview del Corso

M1. Introduzione alla misurazione

M2. Metodi di Misurazione Funzionale

M3. Modelli di qualità del software

M4. Processo sperimentale

M5. Analisi dei dati e Modelli di predizione

- l'approccio sperimentale nell'ambito dell'ingegneria del software

- Processo sperimentale: Definizione; Pianificazione; Esecuzione; Analisi e interpretazione; Presentazione

- Studi empirici: Survey; Casi di studio; Esperimenti controllati

Software Metrics, Analytics, and Quality: Overview del Corso

M1. Introduzione alla misurazione

M2. Metodi di Misurazione Funzionale

M3. Modelli di qualità del software

M4. Processo sperimentale

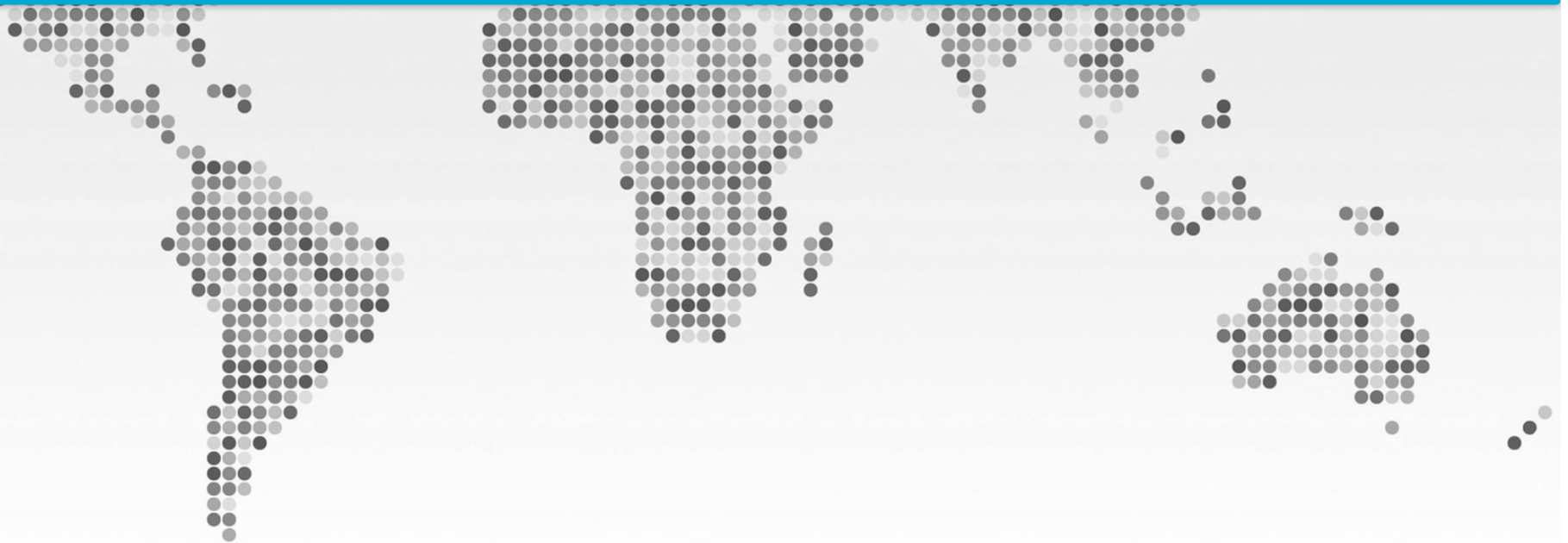
M5. Analisi dei dati e Modelli di predizione

- metodi e strumenti per estrarre e astrarre dati da artefatti software e repository come codice sorgente, sistemi di controllo delle versioni e revisioni, mailing list e discussioni, e issue-tracker e problemi.
- tecniche per analizzare i dati estratti al fine di dedurre l'intento, recuperare comportamenti e processi di sviluppo software da prove o per testare empiricamente ipotesi sullo sviluppo software.
- tecniche per costruire modelli di previsione sfruttando i dati estratti dal software del repository

Carmine Gravino

SeSa Lab and SQM Lab - University of Salerno

Software Metrics, Analytics, and Quality





Carmine Gravino

Docente di: Ingegneria del Software (Triennale L31), Software Metrics, Analytics, and Quality (Magistrale LM-18), IT Project Management (Magistrale LM-66)

E-mail: gravino@unisa.it

Sito web: <https://docenti.unisa.it/004724/home>

Google Scholar: <https://scholar.google.com/citations?user=fGpVbKwAAAAJ&hl=en>

Twitter: [@GravinoCarmine](https://twitter.com/GravinoCarmine)

SeSa Lab Software Engineering @ Salerno

sesalab@unisa.it 

[sesa_lab](#)   

<https://sesalabunisa.github.io/> 

Education

- Ingegneria del Software (x3);
 - Fondamenti di Intelligenza Artificiale;
 - Programmazione Object-Oriented.
 - Gestione dei Progetti Software;
 - Ingegneria, Gestione ed Evoluzione del Software;
 - Metriche e Qualità del Software;
 - Software Engineering for AI;
 - Software Dependability;
 - Didattica dell'Informatica.
-
- 100+ tesisti all'anno;
 - Occasioni di tirocini esterni in azienda.
-
- Più info: <https://sesalabunisa.github.io/it/didattica.html>.



SeSa Lab Software Engineering @ Salerno

sesalab@unisa.it 

[sesa_lab](#)   

<https://sesalabunisa.github.io/> 

Education

- Ingegneria del Software (x3);
- Fondamenti di Intelligenza Artificiale;
- Programmazione Object-Oriented.
- Gestione dei Progetti Software;
- Ingegneria, Gestione ed Evoluzione del Software;
- Metriche e Qualità del Software;
- Software Engineering for AI;
- Software Dependability;
- Didattica dell'Informatica.
- 100+ tesisti all'anno;
- Occasioni di tirocini esterni in azienda.
- Più info: <https://sesalabunisa.github.io/it/didattica.html>.

Research

- Ingegneria dei requisiti;
- Manutenzione ed evoluzione del software;
- Fondamenti di Data Science e Machine Learning per l'Ingegneria del Software;
- Sicurezza software;
- Verifica e convalida del software;
- Aspetti umani e gestione dei progetti software, inclusi sistemi ML e IoT.
- Quantum Software Engineering;
- Accessibilità e usabilità del software;
- Sviluppo del software sostenibile.
- Più info: <https://sesalabunisa.github.io/it/index.html>.

SeSa Lab Software Engineering @ Salerno

sesalab@unisa.it 

[sesa_lab](#)   

<https://sesalabunisa.github.io/> 

Just

Stefano
SeSa Lab - Uni
Fisciano
s.lambiase7@

ABSTRACT

During software changes that are strict deadlines. A sacrificed, possibly sign or implement studies have shown source code mon and causes great developers. Refactor remove code sm source code. How automated, often code instead of a toring operations throughout the p ing, in this paper is able to detect automatically.

Tool, JetBrains: h
Github, <https://g>
Video, <https://yo>

KEYWORDS

Code smells, Ref
ACM Reference F
Manuel De Stefano
Fabio Palomba, An
Code Smell Detect
International Confe
York, NY, USA, 5 p

1 INTRODUCTION

Even after its rele to be enhanced, users [10]. Unfor

Permission to make digital classroom use is granted for profit or commercial on the first page. Copyrights must be honored. Abstracting to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permission from ACM ISBN 978-3-xxxx-xxxx-xxxx/doi.org/10.1145/1122445.1122456

Refactoring Android-specific Energy Smells: A Plugin for Android Studio

Emanuele Iannone
SeSa Lab - University of Salerno
Fisciano (SA), Italy
e.iannone16@studenti.unisa.it

Fabiano Pecorelli
SeSa Lab - University of Salerno
Fisciano (SA), Italy
fpecorelli@unisa.it

Dario Di Nucci
JADE Lab - University of
Tilburg/JADS
's-Hertogenbosch, The Netherlands
d.dinucci@uvt.nl

Fabio Palomba
SeSa Lab - University of Salerno
Fisciano (SA), Italy
fpalomba@unisa.it

Andrea De Lucia
SeSa Lab - University of Salerno
Fisciano (SA), Italy
adelucia@unisa.it

ABSTRACT

Test smells represent s tions applied when de shown that these smel effectiveness of tests at methods to automatic lack of tools that develo ment environment t this paper, we present Smells), an IntelliJ plug detection mechanism t i.e., General Fixture, Ea ods, at commit-level a through the integrated

Video, <https://youtu.be/...>
Source Code, <https://g>

CCS CONCEPTS

• Software and its e tools.

KEYWORDS

Test Smells, Refactoring

ACM Reference Format

Stefano Lambiase, Andre and Fabio Palomba, 2018 toring: The DARTS Proj Conference on Program C USA, 5 pages. <https://doi.org/10.1145/1122445.1122456>

ABSTRACT

Mobile applications are major means to perform daily actions, including social and emergency connectivity. However, their usability is threatened by energy consumption that may be impacted by code smells i.e., symptoms of bad implementation and design practices. In particular, researchers derived a set of mobile-specific code smells resulting in increased energy consumption of mobile apps and removing such smells through refactoring can mitigate the problem. In this paper, we extend and revise ADOCTOR, a tool that we previously implemented to identify energy-related smells. On the one hand, we present and implement automated refactoring solutions to those smells. On the other hand, we make the tool completely open-source and available in ANDROID STUDIO as a plugin published in the official store. The video showing the tool in action is available at: <https://www.youtube.com/watch?v=1c2EhVXIKg>

CCS CONCEPTS

• Software and its engineering → Software maintenance tools.

KEYWORDS

Code smells, Refactoring, Energy Consumption.

ACM Reference Format:

Emanuele Iannone, Fabiano Pecorelli, Dario Di Nucci, Fabio Palomba, and Andrea De Lucia, 2018. Refactoring Android-specific Energy Smells: A Plugin for Android Studio. In *Proceedings of Seoul '20: K/PC International Conference on Program Comprehension (Seoul '20)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/1122445.1122456>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permission from ACM ISBN 978-3-xxxx-xxxx-xxxx/doi.org/10.1145/1122445.1122456

1 Best Paper Award
International Conference on Software Code, Analysis, and Manipulation

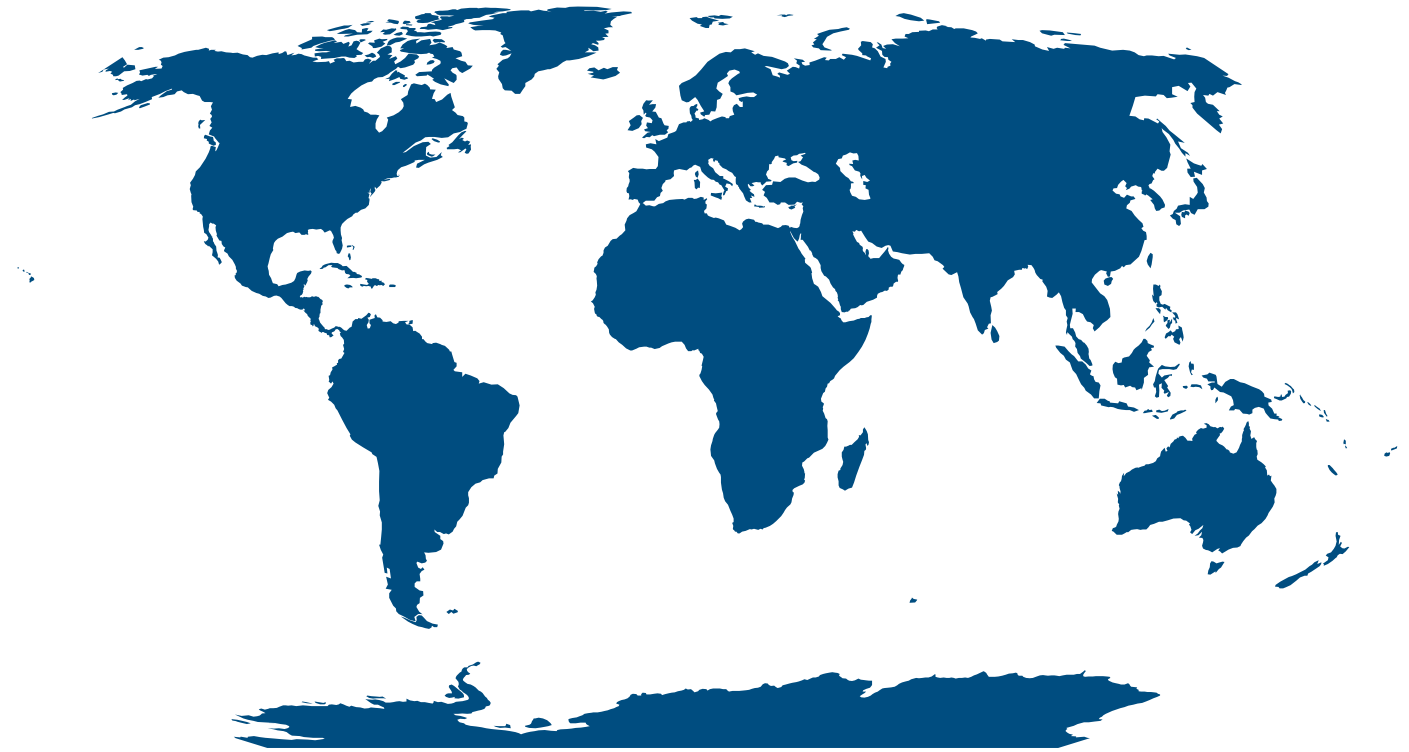
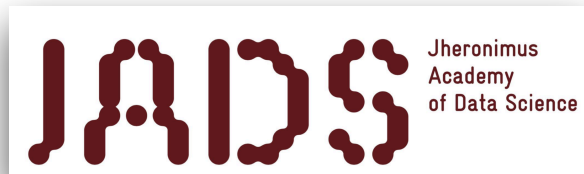
1 ACM/SIGSOFT Distinguished Paper Award
International Conference on Automated Software Engineering

30+ articoli scientifici con studenti in conferenze e riviste internazionali

14 nuovi docenti negli ultimi 10 anni di attività

<https://sesalabunisa.github.io/it/index.html>

II Software Engineering (SeSa) Lab - Collaborazioni Recenti



E altri...

Informazioni amministrative generali

Docente: Carmine Gravino

Orario Lezioni: Lunedì: 11:00 - 13:00 - Aula P6;
Martedì: 12:00 - 14:00 - Aula F5
Mercoledì: 12:00 - 14:00 - Aula F4;

Piattaforma E-learning: <http://elearning.informatica.unisa.it/el-platform/course/view.php?id=819>

Email: gravino@unisa.it

Orario di risposta alle email: Lunedì—Venerdì 12:30 - 13:30

Orario di ricevimento: Lunedì 9:00-11:00; Mercoledì 10:00 - 12:00

Appuntamento sempre richiesto.

Informazioni amministrative - Comunicazione

Destinatario

gravino@unisa.it

Oggetto

[MQS] <Oggetto esplicito, breve e conciso>

Corpo

<Saluto>,

<presentazione del mittente>

<chiara e sintetica descrizione della richiesta/comunicazione>

<segnalazione di eventuali allegati>

<Saluto>

<Firma>

Altre semplici avvertenze e suggerimenti:

- (1) La punteggiatura è importante, usatela appropriatamente;
- (2) Usate le lettere maiuscole, il grassetto e l'italico appropriatamente;
- (3) Siate ordinati, ad esempio dividendo il corpo in paragrafi;
- (4) Prima di inviare l'email, verificate la presenza di errori ortografici e/o di battitura;
- (5) Date del Lei, non del Voi (il periodo fascista è finito 80 anni fa!)
- (6) Le e-mail che non seguiranno il formato riportato **NON verranno considerate!**

Informazioni amministrative - Comunicazione

Destinatario

gravino@unisa.it

Ricorda di rispondere a tutti
(funzione reply-all)

Oggetto

[MQS] <Oggetto esplicito, breve e conciso>

Corpo

<Saluto>,

<presentazione del mittente>

<chiara e sintetica descrizione della richiesta/comunicazione>

<segnalazione di eventuali allegati>

<Saluto>

<Firma>

Altre semplici avvertenze e suggerimenti:

- (1) La punteggiatura è importante, usatela appropriatamente;
- (2) Usate le lettere maiuscole, il grassetto e l'italico appropriatamente;
- (3) Siate ordinati, ad esempio dividendo il corpo in paragrafi;
- (4) Prima di inviare l'email, verificate la presenza di errori ortografici e/o di battitura;
- (5) Date del Lei, non del Voi (il periodo fascista è finito 79 anni fa!)
- (6) Le e-mail che non seguiranno il formato riportato **NON verranno considerate!**

Informazioni amministrative - Comunicazione

Destinatario

gravino@unisa.it

Ricorda di rispondere a tutti
(funzione reply-all)

Oggetto

[MQS] <Oggetto esplicito, breve e conciso>

Corpo

<Saluto>,

<presentazione del mittente>

<chiara e sintetica descrizione della richiesta/comunicazione>

<segnalazione di eventuali allegati>

<Saluto>

<Firma>

Se non ricevi risposta entro 48h,
invia nuovamente la mail!

Altre semplici avvertenze e suggerimenti:

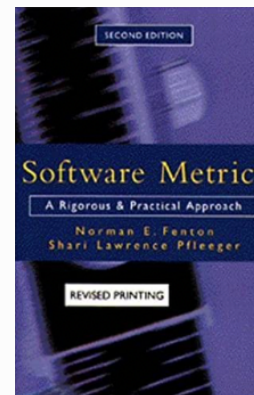
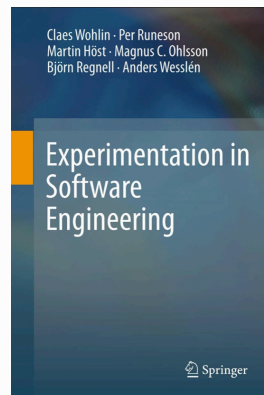
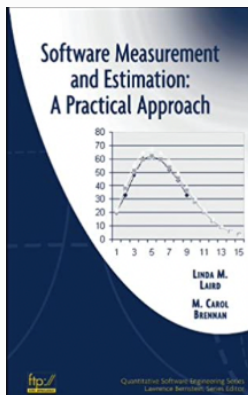
- (1) La punteggiatura è importante, usatela appropriatamente;
- (2) Usate le lettere maiuscole, il grassetto e l'italico appropriatamente;
- (3) Siate ordinati, ad esempio dividendo il corpo in paragrafi;
- (4) Prima di inviare l'email, verificate la presenza di errori ortografici e/o di battitura;
- (5) Date del Lei, non del Voi (il periodo fascista è finito 79 anni fa!)
- (6) Le e-mail che non seguiranno il formato riportato **NON verranno considerate!**

Libri suggeriti

L.M. Laird, M.C. Brennan, Software Measurement and Estimation: A Practical Approach, IEEE Computer Society Press, 2006 - ISBN: 0471676225

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, Anders Wesslén - Experimentation in Software Engineering: An Introduction - The Kluwer International Series in Software Engineering

Norman E. Fenton, Shari Lawrence Pfleeger - Software Metrics by - Pws Pub Co.



Slide e materiale aggiuntivo

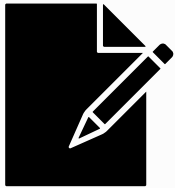
Disponibile sulla piattaforma e-learning del corso: <http://elearning.informatica.unisa.it/el-platform/course/view.php?id=819>

Modalità di Esame

Come ogni esame, ci sarà un accertamento individuale della conoscenza dei contenuti trattati durante il corso. Questa avverrà in due momenti.

Modalità di Esame

Come ogni esame, ci sarà un accertamento individuale della conoscenza dei contenuti trattati durante il corso. Questa avverrà in due momenti.



Prova scritta a fine corso.

La prova sarà composta da domande sui contenuti del corso, la cui valutazione potrà arrivare ad un massimo di 30.

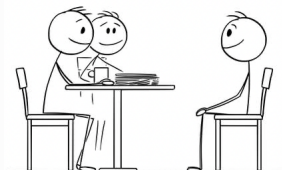
Modalità di Esame

Come ogni esame, ci sarà un accertamento individuale della conoscenza dei contenuti trattati durante il corso. Questa avverrà in due momenti.



Prova scritta a fine corso.

La prova sarà composta da domande sui contenuti del corso, la cui valutazione potrà arrivare ad un massimo di 30.



Discussione della prova scritta e domande sul contenuto del corso.

L'orale è finalizzato ad accertare la conoscenza dei contenuti del corso. La valutazione della discussione potrà arrivare ad un massimo di 30 e abiliterà l'eventuale lode.

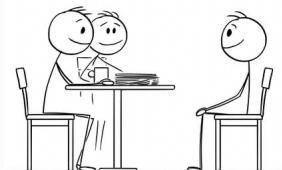
Modalità di Esame

Come ogni esame, ci sarà un accertamento individuale della conoscenza dei contenuti trattati durante il corso. Questa avverrà in due momenti.



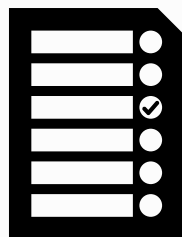
Prova scritta a fine corso.

La prova sarà composta da domande sui contenuti del corso, la cui valutazione potrà arrivare ad un massimo di 30.



Discussione della prova scritta e domande sul contenuto del corso.

L'orale è finalizzato ad accertare la conoscenza dei contenuti del corso. La valutazione potrà arrivare ad un massimo di 30 e abiliterà l'eventuale lode.



Votazione finale.

In linea generale, una media tra le valutazioni delle due prove.

Carmine Gravino

SeSa Lab and SQM Lab - University of Salerno

Software Metrics, Analytics, and Quality

